Documento de análisis estructural y justificación de estructura de datos

Título: Sistema de gestión de almacenes automatizados

Lenguaje: C++

Asignatura: Estructura de datos

Estudiante: Martin Felipe Rodriguez Polania

Carrera: Ingeniería mecatrónica

Docente: Jonatan Gomez Perdomo

Planteamiento del problema: En la industria de la logística y la mensajería actual, los centros de distribución requieren gestionar de manera efectiva el flujo dinámico de los productos con los que trabajan. A pesar del avance tecnológico, en muchos lugares todavía se siguen usando sistemas físicos o parcialmente manuales, disminuyendo enormemente la eficiencia del sistema, generando cuellos de botella, errores en la categorización de envíos, entre otros.

El problema radica en que los sistemas físicos o parcialmente manuales de organización dificultan en gran medida el control, la trazabilidad y la actualización inmediata de la información. Por lo que, cada vez que se quiere hacer cambios en la prioridad de un pedido, su tamaño, su estado actual o incluso agregar algún elemento nuevo en la lista puede convertirse en un trabajo tedioso y propenso a fallas humanas. Asimismo, la auditoria del inventario y el seguimiento del estado del almacén demandan un esfuerzo operativo considerable, pudiendo perder datos existentes y gastando recursos valiosos que podrían destinarse a aumentar la eficiencia del proceso.

Ante esta situación, surge la necesidad de desarrollar una solución capaz de digitalizar y automatizar el flujo operativo dentro del almacén. Mediante el uso de estructuras de datos lineales, el sistema de gestión de almacenes automatizados optimiza el procesamiento de datos, garantizando una administración precisa, ágil y confiable

Objetivos

General: desarrollar un sistema de software interactivo en C++ para la gestión y automatización de un almacén robotizado aplicando estructura de datos lineales para optimizar los procesos de recepción, almacenamiento, despacho y auditoria.

Específicos:

- Implementar la clasificación automática de pedidos mediante una cola con prioridad basada en el nivel de urgencia
- Simular el almacenamiento en estanterías verticales y la función de reversión de comandos utilizando pilas
- Garantizar un balance equitativo de carga hacia la flota de transporte mediante una cola circular
- Mantener un registro histórico accesible en un arreglo dinámico con capacidad de persistencia de datos mediante archivos planos

Justificación teórica y mapeo de estructuras

1. Cola con prioridad: Esta estructura se utiliza para modelar la fila de entrada de paquetes pendientes por procesar dentro del almacén. Cada paquete ingresado cuenta con una etiqueta de prioridad, de modo que se garantiza que el elemento con el nivel de urgencia más alto sea desencolado y atendido en primer lugar. Superando el orden cronológico de llegada para responder de forma ágil a solicitudes de reabastecimiento prioritarias.
2. Pila: Esta estructura se implementa tanto para simular la columna de almacenamiento físico en las estanterías del almacén como para gestionar el historial de acciones del operador. Refleja la restricción física de una estantería vertical donde solo se tiene acceso directo al producto ubicado en la cima; al mismo tiempo, el almacenamiento secuencial de comandos en una pila permite revertir la última operación efectuada mediante un desapilado directo.
3. Cola circular: Esta estructura se emplea para distribuir equitativamente las tareas de despacho de paquetes procesados entre una flota fija de N unidades robóticas, La cola circular permite iterar de manera continua sobre el conjunto disponible de operadores en O(1) reutilizando las posiciones del arreglo al alcanzar el limite final sin necesidad de realizar reasignaciones de memoria ni desplazar elementos
4. Arreglo dinámico: Esta estructura se utiliza para mantener el registro contiguo de todo el inventario de productos gestionados dentro del centro de distribución. Gracias a su capacidad para redimensionarse automáticamente a medida que el almacén se expande, la estructura garantiza un acceso aleatorio directo por índice en O(1), lo que resulta fundamental para realizar consultas rápidas, búsquedas eficientes, ordenamientos continuos y la exportación masiva del catálogo hacia archivos externos de persistencia.

Justificación del uso de la biblioteca estándar de C++:
Para el desarrollo del sistema se optó por utilizar los contenedores nativos proporcionados por la Standard Template Library (STL) de C++. Esta decisión de se fundamenta en tres pilares clave para el desarrollo de software seguro y escalable:

1. Eficiencia algorítmica garantizada: Las implementaciones de la STL están altamente optimizadas a nivel de compilador. El uso de std::priority_queue asegura matemáticamente un rendimiento de O(log n) en inserciones mediante estructuras de heap subyacentes mientras que std::stack y std::vector garantizan accesos y modificaciones en tiempo constante O(1).
2. Seguridad y prevención de fugas de memoria: Al delegar la gestión de la memoria dinámica a los contenedores estándar se mitiga por completo el riesgo de errores críticos como fugas de memoria o accesos a memoria no permitida. Los destructores automáticos de la STL liberan los recursos de forma segura al finalizar su ciclo de vida
3. Enfoque en la lógica del negocio: En el ámbito industrial la prioridad del desarrollo recae en resolver la problemática operativa. Utilizar librerías probadas y estandarizadas permite concentrar el esfuerzo de programación en la arquitectura del sistema logístico, las reglas de negocio y la interfaz de usuario evitando la reescritura de código fundamental ya resuelto y estandarizado por el propio lenguaje

Persistencia y flujo de datos: Para garantizar la conservación de la información entre diferentes sesiones operativas, la persistencia y el flujo de datos del sistema se gestionan mediante la integración directa de archivos planos. Al iniciar la aplicación, el sistema emplea flujos de entrada para leer automáticamente un archivo, importando el historial de productos y poblando los contenedores de la memoria principal para reconstruir el estado del almacén. Durante la ejecución todas las transacciones operativas como el triaje, almacenamiento y despacho modifican las estructuras en tiempo real; finalmente, al seleccionar la opción de cerrar el sistema, el programa utiliza flujos de salida para iterar sorbe el catálogo global y exportar de forma masiva el estado actualizado de vuelta al archivo de texto, asegurando así una sincronización integral que previene cualquier perdida de información logística